33. Convex Optimization Problems

"Convex Optimization: Getting to the Bottom of the Bowl"

Previous Section:

32. Gradient Descent Methods

Contents:

This section moves us into something deeper. Something more mathematical. And if you've followed along carefully, you might start to notice a pattern across everything we've built so far. Machine Learning. Deep Learning. Neural Networks. They may look different on the surface. Different architectures. Different philosophies. Different ways of learning.

But underneath all of them, there is one common goal: ***They are always trying to optimize something.***

Not in a vague sense. Not in a philosophical sense. But in a very precise, mathematical way. A model is always adjusting itself to either minimize a loss, or maximize a reward. Every weight update, every gradient step, every improvement you've seen so far, comes back to this single idea.

So instead of thinking: “We are building intelligent systems”. I want you to shift your perspective slightly. We are solving optimization problems. And not just any optimization problems, but a very special class that makes everything work smoothly, predictably, and efficiently.

Convex optimization.

In this section, we begin by grounding that idea properly.

1. Define the Convex Optimization Problems

Now we move into something that quietly powers almost everything you’ve seen so far. Optimization.

You’ve already trained models. You’ve seen loss decrease. You’ve watched predictions improve. But underneath all of that, there is always a single question being asked: **“How do we find the best possible solution?”**

This is where Convex Optimization comes in. Convex optimization is not just another mathematical concept. It is one of the reasons many Machine Learning algorithms actually work in practice. Because it gives us something incredibly valuable: **A guarantee.**

If a problem is convex, then any local minimum you find is also the global minimum. In other words, there is no better solution hiding somewhere else. No traps. No misleading paths. Once you reach the bottom, you are done.

To understand this, imagine standing in a valley. Not a chaotic mountain range with multiple dips and peaks. But a perfectly shaped valley, smooth and curved, like a bowl. No matter where you start, whether you are near the edge or halfway down, if you keep walking downhill, you will always end up at the same lowest point. There is only one bottom. This is exactly how convex problems behave.

Now compare that to a rough landscape filled with hills and valleys. You might walk downhill and reach a low point, only to realize later that there was a deeper valley somewhere else. That is what happens in non-convex optimization. You can get stuck in a “good enough” solution that is not actually the best.

Convex optimization removes that uncertainty completely. Formally, a function is convex if, for any two points on its graph, the straight line connecting them lies above or exactly on the graph. But instead of focusing on the definition,

focus on the shape: It curves upward → It forms a single valley → It has one lowest point.

And that lowest point is the optimal solution.

This is why convex functions are so important. They turn optimization into something stable and predictable. Algorithms like Gradient Descent can simply follow the slope downward, step by step, without worrying about getting trapped in the wrong place. And this is why convex optimization appears everywhere:

- Machine Learning models
- Data science problems
- Statistical estimation
- Operations research

Because when a problem is convex, we are not just searching for a solution. We are guaranteed to find the best one.

2. Convex Function and Concave Function

Now let's sharpen the picture. In the previous section, you saw the idea of a "valley." Here, we make that idea precise by separating two fundamental shapes you will see again and again: Convex and Concave.

But instead of jumping straight into formulas, let's begin with intuition.

- If you are standing in a valley shaped like a bowl, every step you take downhill leads you toward the same lowest point. There are no hidden dips. No misleading turns. Just one clear destination. That is a **convex function**.
- On the other hand, imagine standing on top of a hill shaped like an upside-down bowl. Every step you take downhill moves you away from the highest point. If your goal was to reach the top, you would need to move upward instead. That is a **concave function**.

So before anything else, fix this in your mind:

- Convex → curves upward like a bowl → one global minimum
- Concave → curves downward like an upside-down bowl → one global maximum

This difference may look simple, but it completely changes how we solve problems.

Convex functions are ideal when our goal is **minimization**, which is exactly what we do in Machine Learning. Loss functions, error measurements, cost functions — all of them are designed so we can move downward and reach the best solution safely.

Concave functions, on the other hand, are often used when the goal is **maximization**. You will see them in economics, utility functions, and profit optimization, where the objective is to reach the highest possible value.

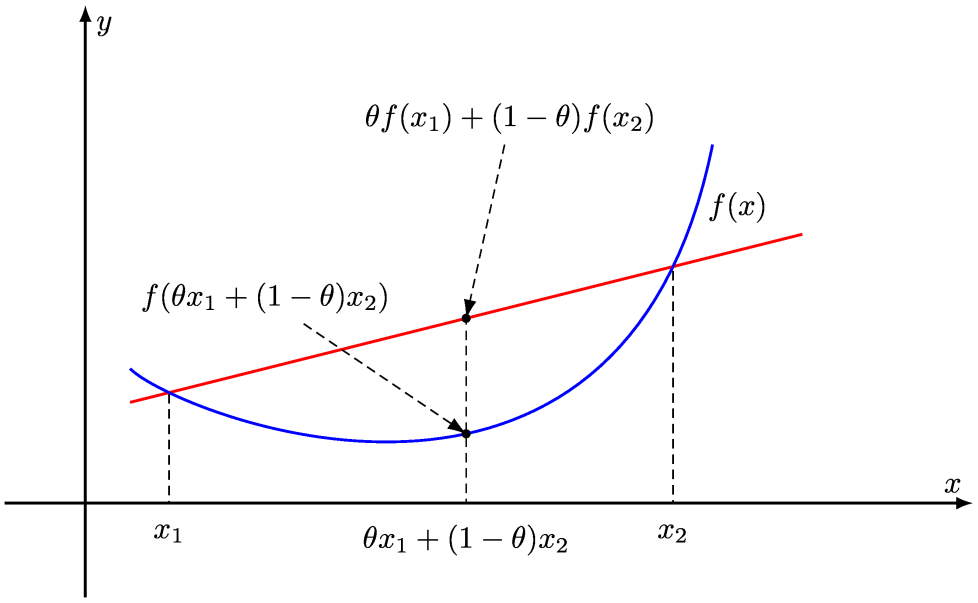
Now here is the important part. Understanding whether a function is convex or concave is not just theory. It directly determines whether our optimization process will succeed or fail.

If the function is convex, methods like Gradient Descent become reliable. You can simply follow the slope downward, step by step, and you are guaranteed to reach the best possible solution. There is no risk of getting trapped in the wrong place.

To make this idea precise, we need a mathematical definition. A function f is convex if it satisfies the following condition:

$$f(\lambda x_1 + (1-\lambda)x_2) \leq \lambda f(x_1) + (1-\lambda) f(x_2)$$

This expression may look abstract at first, but it describes something very visual. ***(Reference the image below):***



Think of two points on the curve: x_1 and x_2. Now connect these two points with a straight line. Here is how to interpret the inequality:

- The left side represents a point on the curve between x_1 and x_2
- The right side represents a point on the straight line connecting them

For a convex function, the curve always lies **below or exactly on** that straight line. That is the “bowl” shape, expressed mathematically.

Now let’s understand the role of λ, because it controls everything here. The parameter $\lambda$is a mixing coefficient. It tells us where we are between the two points:

- $\lambda = 0$ → we are exactly at x_2
- $\lambda = 1$ → we are exactly at x_1
- $0 < \lambda < 1$ → we are somewhere in between

As λ moves from 0 to 1, the point $\lambda x_1 + (1-\lambda) x_2$ slides smoothly along the line segment connecting x_2 to x_1.

And here is the key idea I want you to see: *For a convex function, no matter where you stand between two points, the curve will never rise above the straight path connecting them. It always stays “under control”.* That single property is what makes convex optimization so powerful. It removes uncertainty. It guarantees structure. And most importantly, it ensures that when

we search for a solution, we are not guessing. We are moving toward something we know we can reach.

3. Global Minimum and Local Minimum

Now we reach a question that quietly determines whether optimization succeeds, or fails. You've heard the terms before: Global minimum. Local minimum. But what do they actually mean when you are standing inside the landscape of a function?

Let's go back to the valley. Imagine a large terrain with multiple dips. Some are shallow. Some are deep. As you walk downhill, you might reach a point where every direction around you goes upward. You stop there, thinking you've reached the lowest point.

But have you? Not necessarily. That point is a **local minimum**. It is the lowest point in its immediate neighborhood. But somewhere else in the landscape, there could be a deeper valley. That deeper point is the **global minimum**.

So let's make it clear:

- **Local minimum** → lowest point in a small region
- **Global minimum** → lowest point across the entire function
- And a convex function is a function such that ***the local minimum is also the global minimum.***

Now here comes the rule that often confuses people: There can be many local minima. But there is only one global minimum. And another question comes up:

Is the global minimum also a local minimum?

Yes. Always. Because if a point is the lowest in the entire function, it is automatically the lowest in its neighborhood as well. So the global minimum is a special case of a local minimum. It is the *best* one.

Now let's refine another question:

Is the global minimum the minimum point, or the minimum value?

The answer is: It can refer to both, depending on what you mean. Let's make it precise.

Suppose we are solving the problem: Find x such that $y = f(x)$ is as small as possible.

There are two things happening here:

- The **value** $y = f(x)$ becomes as small as possible → this is the **minimum value**
- The **input** x that achieves this value → this is the **minimum point** (also called the *argmin*)

So we distinguish them clearly:

- **Minimum value** → the smallest output of the function
- **Minimum point (argmin)** → the input where that smallest value occurs

For example, if a function reaches its lowest value at $x = 2$, and $f(2) = 1$, then:

- Minimum point = $x^* = 2$
- Minimum value = $f(x^*) = 1$

So when we say "global minimum," we must be careful with language. Sometimes it refers to the value, sometimes to the point. But conceptually, it always means: ***The lowest value of the function, and the point where it is achieved*** → Almost always, it refers to the minimum value.

Now step back and connect this to everything you've learned so far. Optimization is the process of finding the best possible value of a function. Sometimes we minimize. Sometimes we maximize.

But in Machine Learning, we almost always minimize a loss. And this is where convexity becomes powerful. If the function is **convex**, everything becomes simple:

- There is only one global minimum
- There are no misleading local minima
- Any descent method will reach the correct solution

But if the function is **non-convex**, the landscape becomes dangerous:

- Multiple local minima exist
- Algorithms can get stuck

- The final solution may not be optimal

This is why convex optimization is so important. It removes the uncertainty.

Now let's look at what this gives us in practice. When working with convex functions, we gain three major advantages:

- **Global Minimum Assurance:** Any local minimum found is guaranteed to be the global minimum. There is no ambiguity in the result.
- **Faster and Stable Convergence:** Algorithms like Gradient Descent can move smoothly downhill without needing tricks to escape bad regions.
- **Simplified Analysis:** The mathematics becomes cleaner. The behavior becomes predictable. Both theory and implementation become easier.

And this is not just theory. You've already been using this idea, even if you didn't notice.

In Machine Learning, training a model means minimizing a loss function. Many of these loss functions are designed to be convex so that optimization is stable and reliable.

You've seen one of them: ***Mean Squared Error (MSE)***. But it's not the only one. Other important convex loss functions include:

- **Mean Absolute Error (MAE)** → robust to outliers
- **Huber Loss** → combines MSE and MAE for stability
- **Log Loss (Binary Cross-Entropy)** → used in logistic regression
- **Hinge Loss** → used in Support Vector Machines

And because of these convex properties, many core models in Machine Learning rely on convex optimization: Linear Regression, Logistic Regression, Support Vector Machines (SVM)

So when you train these models, you are not just "fitting data". You are solving a convex optimization problem where the destination is guaranteed.

And that changes everything. Because now, optimization is no longer a blind search. It becomes a guided descent, toward a solution you know exists.

4. Example: Checking Convexity and Finding the Minimum

Now let's stop thinking abstractly for a moment, and actually *touch* the math. Because up to now, convexity sounded intuitive. A valley. A bowl. A shape you can imagine. But I do not want you to just imagine it. I want you to *verify it*.

Step 1: Check for Convexity

Consider the function: $f(x) = x^2 + 1$

To determine whether this function is convex, we examine its derivatives.

$$f(x) = x^2 + 1, \quad f'(x) = 2x, \quad f''(x) = 2$$

Now look at the second derivative. It is constant. It is positive. It does not depend on x.

So we conclude:

- $f''(x) > 0$ for all x
- The function curves upward everywhere
- Therefore, the function is **convex on the entire real line**

This is the analytical way of confirming what your intuition already suspected.

Step 2: Find the Minimum

Now that we know the function is convex, finding the minimum becomes straightforward.

We take the first derivative: $f'(x) = 2x$

Set it equal to zero: $2x = 0 \Rightarrow x = 0$

Now evaluate the function at this point: $f(0) = 0^2 + 1 = 1$

So we get:

- Minimum point (argmin): $x^* = 0$
- Minimum value: $f(x^*) = 1$

And because the function is convex: This local minimum is also the **global minimum**. No doubt. No alternatives. No hidden valleys.

Convex Optimization Using Python

Now we shift from theory, to something more experimental. Instead of using derivatives, we can *numerically test* convexity using the definition you learned earlier.

The idea is simple: Pick two points x_1 and x_2, then check whether the convexity inequality holds for values of $\lambda \in [0, 1]$. If it ever fails, the function is not convex on that interval.

Here is the implementation:

```
import numpy as np
import matplotlib.pyplot as plt

def is_convex_function(f, x1, x2, iterations=100, tolerance
=1e-9):
    # Lambda is always between 0 and 1
    lambdas = np.linspace(0, 1, iterations)

    for lamb in lambdas:
        f_left = f(lamb * x1 + (1 - lamb) * x2)
        f_right = lamb * f(x1) + (1 - lamb) * f(x2)

        # If inequality is violated → not convex
        if f_left > f_right + tolerance:
            return False

    return True
```

Now the main program:

```
f_input = input("Enter equation f(x): ").replace("^", "**")
f = eval(f"lambda x: {f_input}", {"np": np})

x1, x2 = eval(input("Enter x1, x2: "))

if is_convex_function(f, x1, x2):
    print(f"The function is convex with x1 = {x1}; x2 = {x
2}")
else:
    print("The function is not convex")
```

Enter equation f(x): x^4-x^2
Enter x1, x2: 2,3
The function is convex with x1 = 2; x2 = 3

Enter equation f(x): x^4-x^2
Enter x1, x2: -0.5,0.5
The function is not convex

When Convexity Depends on the Interval

Now here is where things get interesting. Consider the function: $f(x) = x^4 - x^2$

This function is **not globally convex**. But your program might still return different results depending on the interval you choose.

Case 1: Interval $[2, 3]$

- The convexity condition holds for all sampled \lambda
- Output: **True**

On this interval, the function behaves like a convex function. The curvature is positive, so it “looks” like a bowl.

Case 2: Interval $[-0.5, 0.5]$

- The convexity condition is violated for some \lambda
- Output: **False**

Here, the function curves downward near the origin.

We can confirm this analytically: $f''(x) = 12x^2 - 2$

Near $x = 0$, this becomes negative → which means **concave behavior**.

What This Actually Means

The same function gives different answers because:

- Convexity can be **interval-dependent**
- Your algorithm only checks between x_1 and x_2
- A function can be convex in one region and concave in another

So keep this in mind: Numerical checks reveal behavior in a region. They do not prove global convexity. To get a better approximation, you can test multiple intervals: $(-10, 10); (-100, 100); (-999, 999)$

But even then, you are still approximating. That is why theory still matters.

Summary

At this point, nothing here is completely new. You already knew about derivatives. You already knew about maxima and minima. But before this, those ideas were isolated. Now they are connected.

Convexity is not just about shape. It is about **guarantee**.

- A convex function gives you one destination
- A zero gradient gives you the candidate point
- A positive second derivative confirms the shape

And suddenly, optimization becomes predictable. What you learned before was: “How to find a minimum” → What you understand now is: “When that minimum is the only one that matters”

And that shift? That is where optimization becomes powerful.

References

https://www.geeksforgeeks.org/machine-learning/convexity-optimization/

https://www.geeksforgeeks.org/engineering-mathematics/convex-and-concave-functions/

https://mdobook.github.io/html/convex/

Next Section:

34. Rule-Based Classifiers